Test Plan Document
SP-14 - Green - Chess AI
CS 4850 - Section 03 – Fall 2025
Advisor: Sharon Perry
Nov 25, 2025

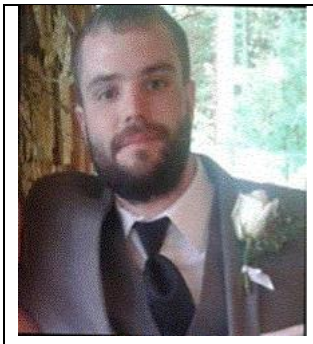	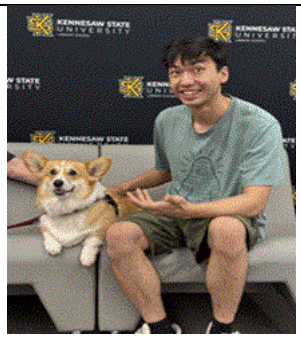	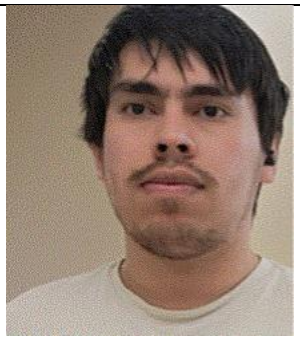	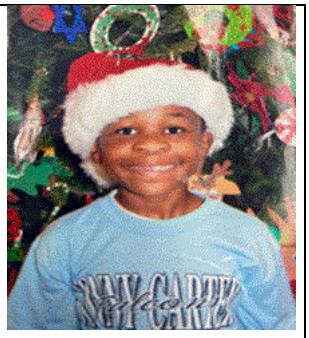
Matt Staats Developer	Jason Nguyen Documentation	Carlos Pena Documentation	Chiagoziem Iteogu Developer

Table of Contents

1. Overview

1.1 Purpose:

This document shows the testing scope and the focus areas and objects for our chess AI web application. It goes over testing strategies and those test responsibilities. Shows and gives examples of tests cases with their expected results. The entry and exit criteria for these testing phases and the documentation of the test results for the Software Test Report.

1.2 Scope

The document shows the details of the testing performed for our chess AI web application. All four game modes will be tested: local h2h, player vs ai, ai vs ai, and online multiplayer. The three ai engines will be tested, real-time multiplayer functionality using WebSocket connections. Testing will be performed and accomplished by manual functional testing and integration testing.

2. Test Objectives

Main Objectives:

2.1 Verify game mode functionality

- All four game modes work as intended and designed.

2.2 AI Integration

- All three AI engines are set up correctly.
- All three produce legal, valid, and reliable moves.

2.3 Multiplayer Reliability

- Game rooms need real-time sync to work correctly and keep the game's state persistent.

2.4 UI Responsiveness

- UI design is to understand
- Themes and styles change work smoothly.

2.5 Test Handling

- Verify that illegal moves are rejected.
- Verify that legal moves are accepted.
- Validate AI move generation when button is clicked.
- Verify the undo button resets the last piece, moves back to the previous position, and the history panel is fixed.
- Verify reset button clears the game
- Test disconnections and resyncs for multiplayer games.
- Check all functionality after every release of new content for full integration testing.

3. Scope of Testing

3.1 In Scope

3.1.1 Chess Functionality

- Move validation of legal moves
- Checkmate detection
- Draw conditions
- Stalemate detection
- Special rules (pawn promotion, en passant, castling).

3.1.2 Game Modes

- Local 2 Player mode
- Player vs AI mode
- Online Multiplayer mode
- AI vs AI mode (simulates full matches in front of the user).

3.1.3 AI Engine Integration

- Custom Minimax algorithm depth of 2 to 3
- Stockfish engine moves and best move evaluation
- Leela Chess Zero engine

3.1.4 Multiplayer Features

- Game ID generation and Room creation
- Chat system with message timestamps
- Player color assignment
- Disconnection and reconnection to game room
- WebSocket real-time synchronization

3.1.5 UI

- Move History panel display
- Evaluation bar of move when playing against AI
- Board themes
- Piece themes and styles
- Board flip functionality

3.1.6 Management of Game State

- JSON file persistence
- Save/load game state
- Chat message persistence

3.2 Out of Scope

- Linux operating system: testing was limited to Windows and macOS only.

- Mobile devices: No testing and compatibility for phones and tablets
- Modern browsers only.
- Touch screens do not allow drag and drop. Best for mouse clicks.
- Multiplayer testing on a large scale with 50+ users playing, 100+, etc.
- No AI difficulty scaling
- Manually join multiplayer games, no ranking or matchmaking system for players.
- No player stats kept for historical data on user's past games.
- No player profiles
- No AI engine comparison and tracking to improve our custom minimax algorithm.

4. Test Strategy

4.1 Manual Functional Testing

The team will be testing the web application directly to validate the application's functionality and its correctness. Including things like the UI, the four game modes, and AI move generation and rule enforcement.

4.2 Integration Testing

Our team will test the interactions between all game engines, and all three AI engines, and the multiplayer system. Testing to make sure the game state is persistent, chat is synchronized and working, and each integration is working correctly across all the components of the application.

4.3 Regression Testing

The team will test after each update and feature that is added. Using back testing to test that each past update still works, and previous features remain unaffected by the new feature. Especially, focusing on the key functionalities is still intact and functional.

4.4 Test Case Prioritization

The team will focus on test cases with high or very high priority as these relate to functional requirements. Things like core game play rules, end game detection, and AI move generation. Cases will be tested manually in certain scenarios, and the tester will document and observe the results.

5. Test Cases

5.1 Local 2 Player Mode Test Cases

Test ID	Test Objective	Action / Procedure	Expected Result	Actual Result	Priority	Pass/Fail
LM01	Valid piece move	- Try all different pieces for valid piece placements.	Every piece should follow their movement rules and only move to valid spots.	Valid moves were all accepted for every chess piece.	Very High	Pass
LM02	Illegal move rejection	- Try illegal move	The piece will be moved back to its current position, rejecting the invalid move.	Move was rejected and the piece was not moved.	Very High	Pass
LM03	Turn validation	- try to move black out of turn - try to move white out of turn	White and black pieces should only move when it's their turn.	Turn validation is correct.		
LM04	Checkmate detection	-play game till checkmate position	The game ends and the board is now locked.	Game ends and board is locked.	Very High	Pass
LM05	Stalemate detection	-play till stalemate position	The game ends and the board is now locked.	Game ends and board is locked.	Medium	Pass

LM06	Check detection	-play till check position	Game is locked till a move, moves the user out of check.	Players have to make valid moves out of check.	Very High	Pass
LM07	Special rules work and valid (castling, double pawn step, etc.)	-play till all special rules	Castling when valid, double pawn step and en passant when valid.	Special rules all work in valid situations.	High	Pass
LM08	Pawn promotion	-play till pawn promotion	A pawn should be promoted to the queen.	The pawn is correctly promoted.	High	Pass
LM09	Reset game control	-make a move - hit reset button	The board should reset back to the start.	Chessboard correctly resets.	High	Pass
LM010	Undo move control	-make a move - hit undo button	The last move made should be reset back to the previous position, and the history panel updated.	The last piece is correctly reset, and the history panel is updated.	Medium	Pass
LM11	Flip Board	-hit the flip board button	The board should switch sides. Moving the black pieces to bottom and white to the top. Or vice versa.	The board correctly flips positions.	Low	Pass

5.2 Player vs AI Mode Test Cases

Test ID	**Test Objective**	**Action / Procedure**	**Expected Result**	**Actual Result**	**Priority**	**Pass/Fail**
AI01	Stockfish engine piece movement	-start player vs ai -select stockfish	Stockfish engine makes a valid move.	Stockfish makes a valid move.	Very High	Pass

		-hit make engine move button				
AI02	Leela engine piece movement	-start player vs ai -select Leela engine -hit make engine move button	The Leela engine makes a valid move.	Leela makes a valid move.	Very High	Pass
AI03	Minimax engine piece movement	-start player vs ai -select Minimax engine -hit make engine move button	Our custom minimax engine makes a valid move.	Custom minimax makes a valid move.	Very High	Pass
AI04	Evaluation bar updates per move	-start player vs ai - make a move -make an engine move	Evaluation score should display per move, user or ai. And display a score.	A score is displayed on the last move made.	Medium	Pass
AI05	AI engine selection control	-start player vs ai -select different ai engines	Pick any engine from dropdown. And make the engine move.	You can select any AI engine from the list and make the engine move at the users' will.	High	Pass
AI06	AI engine turn control	-Check and make sure AI only moves when button is pressed	AI should not move unless a button is clicked.	AI does not move until the button is pressed.	High	Pass
AI07	Legal move validation	-start player vs ai - check console for illegal moves trying to be made by AI.	AI should not try to move illegally, if so, it's caught and new move is produced.	AI only makes valid moves for its turn.	High	Pass

5.3 Multiplayer Online Mode Test Cases

Test ID	Test Objective	Action / Procedure	Expected Result	Actual Result	Priority	Pass/Fail

MM01	Create a new game with UUID	-select multiplayer game mode - create a new game room.	A new room should be created and set up with a unique generated ID for the room.	A new room was created with a unique UUID.	Medium	Pass
MM02	Join game from lobby list	-select multiplayer mode - click drop down lobby list - select one from the list	The user joins a game room with an open spot.	User successful can join rooms.	Medium	Pass
MM03	Join game with lobby id	-enter lobby id in text field asking for id	Users should be brought to the correct game room with the correct code.	The user is correctly brought to the right room.	Medium	Pass
MM04	Available list of open games in lobby	-select multiplayer mode -select drop down list to view open game rooms.	User clicks drop down of rooms; list is displayed with only games with an open spot.	List is displayed, and full games are hidden.	Medium	Pass
MM05	Player white move sync	-start a game, allow second player to join - start playing -monitor game state, turn validation	Player white move should display on player black's screen and be in sync.	The move player's move shows up on the opponent's screen and is up to date.	High	Pass
MM06	Player black move sync	-start a game, allow second player to join - start playing -monitor game state, turn validation	Player black move should be displayed on player white's screen and in sync.	The player's move shows up on the opponent's screen and is up to date.	High	Pass
MM07	Game State persistency	-start a multiplayer game - finish the game	The game should stay consistent with both users making moves.	The game stays consistent and up to date.	High	Pass

MM08	Reconnect to lobby handling	-have a multiplayer game going -leave and try to reconnect to the same game room.	Users should be able to rejoin the current game room they were in before disconnecting.	Users can reconnect, but the game state is not consistent.	Low	Fail
MM09	Chat messages send and receive	-start or join a multiplayer game - both users send message to test chat	Both users should be able to send and receive messages with the correct timestamps	Chat message back and forth worked for both users in a game room.	Low	Pass
MM10	Correct turn reinforcement	-start a multiplayer game - make sure white and black can only move on correct turn	Players playing white and black should only be able to move when it's their turn.	Turn validation is enforced for both users.	High	Pass

5.4 AI vs AI Mode Test Cases

Test ID	**Test Objective**	**Action / Procedure**	**Expected Result**	**Actual Result**	**Priority**	**Pass/Fail**
AA01	Automated game simulation	-select engines - click start ai vs ai button	Engines take alternate turns making moves automatically.	Engines make moves and play.	High	Pass
AA02	Stop Functionality	-start ai vs ai match - hit the stop button	The match should stop, and the board will reset.	The match stops and the board is reset.	High	Pass
AA03	Play to end game				Medium	Pass
AA04	Update Move History	-start ai vs ai - Watch the move history panel for updates.	History should display the complete turn for white and black.	The history panel displays the move made by AIs.	Low	Pass

AA05	Same three moves in a row force draw scenario	-start ai vs ai - watch move history for three move repeats & game stoppage -repeat if the case is not triggered.	When the same three moves are made in a row by one player, the match should end in a draw.	Match stops and ends in a draw.	Medium	Pass
AA06	Same move sequence three times in a row force draw scenario	-start ai vs ai - watch move history for three sequence repeats. -repeat if the case is not triggered.	When the same move sequence is made by both AIs in row three times. The match should end in a draw.	Match stops and ends in a draw.	Medium	Pass
AA07	Not enough pieces left for capture, force draw scenario	-start ai vs ai - watch games and check console logs for a draw. -repeat if the case is not triggered.	The match should stop and end in a draw, when both players do not have enough pieces to checkmate.	The match stops and ends in a draw.	Medium	Pass
AA08	Same AI engine vs itself	-select the same engine for white and black -start ai vs ai match	The match simulation begins with the AI vs itself	The match starts and it begins playing itself in a game. Any of the 3 engines.	Medium	Pass

5.5 Game State and Persistence Test Cases

Test ID	Test Objective	Action / Procedure	Expected Result	Actual Result	Priority	Pass/Fail
GP01	Game status tracking	-play a game till check or checkmate -check status	Status of game is updated correctly	Status is updated	Very High	Pass
GP02	JSON save and update per move	-make a valid move - check saved folder in project	JSON file is created (if first move) or updated	JSON file is created/updated	Medium	Pass

GP03	Chat persistence	-send message -reload multiplayer game	Chat history has been restored.	Chat history is not restored.	Low	Fail
GP04	Load saved game state	-close app -restart -load game	Game resumes from saved position.	The board is not resumed on rejoining player	Medium	Fail

5.6 UI Customization Test Cases

Test ID	**Test Objective**	**Action / Procedure**	**Expected Result**	**Actual Result**	**Priority**	**Pass/ Fail**
UI01	Changing piece theme color	-select a game mode - change piece theme to a different color	The chess pieces will change for the correct color styles. Should work and be applied to them all.	Every color piece style worked and changed to the correct color patterns selected.	Low	Pass
UI02	Changing board theme color	-select a game mode - change board theme to a different theme.	The board color scheme should change to the selected option. Should work and be applied to them all.	The board correctly changed to the correct theme depending on the option chosen.	Low	Pass
UI03	Toggle different piece styles	-select a game mode - change the piece style to a different style.	The pieces should change to a different style. Currently offering 2.	The chess pieces all changed to whatever style was selected.	Low	Pass
UI04	Move History	-start a game - move pieces and do multiple complete turns for both sides.	The history panel should display the moves in the format of complete turns. White and black piece moves will appear in the same line for a turn.	History is displayed correctly and in the right format.	Low	Pass

UI05	Chat timestamps	-start a multiplayer game. - wait for player 2 to join the lobby. -send messages in chat.	The timestamp for each received message should have the correct time.	Each message displays a timestamp underneath.	Low	Pass

6. Test Procedure

6.1. Local Mode

6.1.1 Legal Moves

- Step 1: Drag a chess piece with the mouse
- Step 2: System checks if the move is legal
- Expected Result: Chessboard updates to reflect the legal move

6.1.2 Illegal Moves

- Step 1: Attempt to play a move out of turn or an illegal move
- Step 2: System checks if the move is illegal
- Expected Result: Board does not update; move is rejected

6.2. Player vs AI

6.2.1 AI Moves

- Step 1: Select Player vs AI mode
- Step 2: Select an AI engine (Stockfish, Leela, Minimax, etc.)
- Step 3: Click the button to generate AI moves
- Expected Result: AI move is applied to the client-side gameboard

6.3. AI vs AI

6.3.1 AI Selection

- Step 1: Select AI vs AI mode
- Step 2: Choose engines for both sides
- Step 3: Start AI vs AI mode
- Expected Result: Each AI generates moves on its respective turn

6.3.2 Game Termination

- Step 1: Allow game to reach a draw or checkmate state
- Expected Result: Game ends and winner/draw is declared

6.4. Multiplayer

6.4.1 Create a Room

- Step 1: Select Multiplayer mode
- Step 2: Create a room
- Step 3: Join as Player 1
- Expected Result: WebSocket connection established to a new saved gameboard on the backend server

6.4.2 Join a Room

- Step 1: Select Multiplayer mode
- Step 2: Enter or select an existing room ID
- Step 3: Join as Player 2
- Expected Result: WebSocket connection established to an existing saved gameboard on the backend server

6.4.3 Chat

- Step 1: Join a room
- Step 2: Enter a name and submit a chat message
- Expected Result: Message is broadcast to all players and saved to the gameboard room's chat history in the backend server

6.5. UI Customization

6.5.1 Customizing Gameboard

- Step 1: Select options such as flip board, board color, or piece style
- Expected Result: Gameboard updates to display selected customization on the client UI

6.5.2 UI interface

- **Step 1:** Click on the website
  Step 2: Verify the display of:
  - Game modes
  - Chessboard UI

    - Move history
    - Customization options
    - Reset / Undo moves features
- **Expected Result:**
    - Website renders correctly.
    - All UI elements are functional.
    - Users can select and play a chess game across different modes.

7. Test Environment

7.1 Software Requirements

Version Control:

- GitHub – Remote repository for storing and managing source code versions.

Web Browsers:

- Google Chrome / Chromium
- Mozilla Firefox

Development Environment:

- IDE or code editor with support for React JavaScript and Python (e.g., VS Code, PyCharm).

Frontend Packages (React/JavaScript):

- react – Core frontend framework
- react-chessboard – UI component for rendering the chessboard
- chess.js – Library for chess logic and move validation

Backend Packages (Python):

- fastapi – Framework for backend API services
- uvicorn – ASGI server for running FastAPI
- python-chess – Chess logic and AI integration
- websockets – Real-time communication between client and server

7.2 Hardware Requirements

Internet Connectivity:

- Required for public/global accessibility, multiplayer connections, and hosting services.

Server Hosting Services:

- Static hosting: GitHub Pages (for frontend deployment)
- Cloud hosting providers for backend APIs (e.g., Render, AWS, Azure)

Local Test Environment:

- Local machine running on Windows or macOS
- Capable of running local development servers (React and FastAPI) and supporting required libraries

8. Test Data

Data requirement

- **File Format:**
  All test data files must be in **JSON format** to ensure compatibility with the system's data parser and backend services.
- **User Move Storage:**
  The data must store each user's chess to move input and accurately reflect **move history information** for all game modes.
- **Message Differentiation:**
  The data layer must distinguish between different **data message types** (e.g., move input, game state update, AI response, chat messages) and process them correctly across multiple game modes.
- **Client Identification:**
  Each data message must include a **client ID** to uniquely identify HTTPS requests from different users.
- **Game State Management:**
  The system must store the **game state** for each client across all game modes (e.g., AI vs player, AI vs AI, online multiplayer).
- **AI Move Integration:**
  The data must support loading and generating **AI moves** from compatible Chess AI engines and store this information as part of the game data.

- **Server Connectivity:**
  The test data must support establishing and validating **web connections** between the client and server backend.
- **Frontend–Backend Data Exchange:**
  Data must be retrievable from the backend and processed by the frontend, including real-time game state updates, session data, and validation results.
- **Real-Time UI Updates:**
  Data processing must enable real-time updates to the **user interface**, such as animations, and chat activity.

9. Assumptions and Dependencies

9.1 Assumptions

- Users have knowledge of how to play the game chess and understand the rules.
- The user's browsers are up to date and properly installed.
- The Leela engine is available to use locally.
- All required dependencies are installed in their project setup (npm, pip, chess.js packages).
- The backend server is running and is accessible to users.
- Users know how to operate and use the UI.

9.2 Dependencies

- Team Coordination: Multiple team members needed to be on to play and test multiplayer capabilities.
- Network Connectivity: Testing the multiplayer requires network connectivity to test if WebSocket connections are functional.
- AI Engines: Leela and Stockfish require proper installation and setup.
- Test Data: Saved game JSON files must be accessible.
- Backend Availability: FastAPI server needs to be running, or testing cannot occur for the web application.

10. Entry and Exit Criteria

10.1 Entry Criteria

Testing conditions to Begin

- All three AI engines are installed, configured, and callable. Including Minimax, Stockfish, Leela.
- WebSocket connections for multiplayer are set up and stable on local tests.
- Testing environment is configured with browser, dependencies installed, local servers running.
- Test data such as JSON game-state files can be created and read.
- All Team members should be available for multiplayer and integration testing when needed.

10.2 Exit Criteria

Testing conditions for completion

- All planned test cases need to be executed
- All required features have been verified with the SRS and meet all the acceptance criteria.
- Any failed Low/Medium priority tests have workarounds or documented impact.
- The build is stable enough for demo, release, or the next development phase.
- Test results should be documented in the software test report.

11. Software Testing Report

Requirement / Feature	Pass	Fail	Severity
Local Mode – Legal Moves			minor
Local Mode – Illegal Moves			minor
Player vs AI – AI Move Generation			Critical
AI vs AI – Move Generation			Moderate
AI vs AI – Game Termination			Critical

Multiplayer – Create Room		Critical
Multiplayer – Join Room		Critical
Multiplayer – Chat		minor
UI Customization – Board & Piece Styles		minor
UI interface feature		Critical

12. Definitions

Term/ Acronym	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
ELO	Chess Rating System
FastAPI	Python Web Framework
FEN	Forsyth-Edwards Notation (used for chess position formatting)
JSON	JavaScript Object Notation
LC0	Leela Chess Zero
PGN	Portable Game Notation (Chess game data format)
STP	Software Test Plan
STR	Software Test Report
UUID	Universally Unique Identifier

13. References

- SP-14-Green-Chess AI- SRS document.